

Wilderfolk Terrain, Season, and Weather

Standalone Visual Prototype Concept

Target: A clean prototype built from scratch in a copy, without changing the current Wilderfolk implementation.

Purpose: Create a readable geographic visual world with coherent shorelines, connected rivers, clustered mountain ranges, seasonal snow, and understandable weather consequences while preserving a future path back into Wilderfolk.

Build the new visual world independently. Keep the current game safe. Integrate only after the new system proves itself.

1. The central idea

The prototype uses two layers. The first is a small invisible logical grid used for all simulation and gameplay functions. The second is a larger visual world used for what the player sees.

Invisible logical grid

→ movement, collision, building checks, production, terrain rules, river cells, map queries, and simulation ticks

Larger visual world

→ terrain sprites, shorelines, rivers, mountain formations, trees, buildings, snow, weather particles, atmosphere, and camera presentation

The logical grid is authoritative. The visual world is a projection of it. A visible mountain formation can cover many logical cells, and a large shoreline sprite can represent many small water, beach, and grass cells. The visual layer must retain a mapping to those cells but must not become a second simulation.

2. Standalone means genuinely clean

The prototype should not copy the current terrain renderer and gradually patch it. It should be a separate small project or copy with its own renderer, visual terrain data, formation logic, and weather layers.

The current Wilderfolk project remains unchanged and serves as the reference version. The prototype may reuse approved art assets, concepts, and the future integration contract, but it should not inherit legacy terrain switches, unused sub-terrain branches, or old cache assumptions by default.

Existing Wilderfolk rules are references, not absolute limits. The prototype may make better decisions from first principles when the old architecture is contradictory or contains unused features.

3. Terrain categories

Only retain a terrain category when it has a real simulation purpose, a meaningful geographic purpose, or a clear player-facing identity.

The starting logical categories are:

```
Grassland
Forest
Beach
RiverBank
ShallowWater
River
DeepWater
Rocky
Mountains
```

`Forest` and `DarkForest` become one `Forest` category when they have no distinct gameplay or visual purpose. `Snow` is not a terrain category. Snow is a seasonal weather and visual condition. `Hills` should remain separate from `Rocky` or `Mountains` only if it has a distinct rule or clearly different visual meaning.

The prototype should use one terrain-definition table for all logical properties:

```
interface TerrainDefinition {
  isWater: boolean;
  isBuildable: boolean;
  movementCost: number;
  visualFamily: 'grass' | 'forest' | 'beach' | 'water' | 'rock' | 'mountain';
  riverRole: 'none' | 'bank' | 'channel';
  formationRole: 'none' | 'mountain';
  efficiency: Partial<Record<BuildingType, number>>;
}
```

This prevents generation, building placement, production, and rendering from using different definitions of the same terrain.

4. Generation pipeline

Generation has two distinct passes.

Logical pass

The generator creates the fine grid from a seed and preset. It produces final terrain categories, water, rivers, forest, shore, and high terrain. It guarantees a buildable starting area without destroying natural water or mountain geography.

Temporary generation values such as noise, moisture, smoothing values, and elevation may be used internally. If they do not affect simulation after generation, they are generation inputs rather than permanent gameplay state.

Rivers and mountain regions are stored as canonical formations. The primary river must be included in the same canonical river data used by the visual renderer and future gameplay systems.

Visual pass

The visual pass groups logical cells into large visible materials and formations. It creates visual formations such as shorelines, river segments, and mountain ranges. Each formation records which logical cells it covers.

```
interface VisualFormation {  
  id: string;  
  kind: 'terrain' | 'shoreline' | 'river' | 'mountain';  
  x: number;  
  y: number;  
  width: number;  
  height: number;  
  coveredCells: number[];  
  spritePath: string;  
  orientation: 'canonical' | 'mirrored';  
}
```

5. Geographic visual language

The prototype should use a small set of strong visual materials rather than many weak near-duplicates.

Materials: grass, forest ground, beach, shallow water, river water,
deep water, rocky ground, mountain formation

Formations: smooth shoreline, connected river, riverbank, mountain range,
mountain peak or landmark

Overlays: winter snow, bank frost, tree-cap snow, roof snow,
mountain-cap snow, drought dryness, fog atmosphere

Shorelines

The visual system must support clear transitions between grass, beach, shallow water, deep water, riverbanks, and river channels. It must not fall back across large regions simply because an adjacent category has no atlas tile. Unsupported boundaries receive a deliberate edge treatment.

Rivers

Rivers are connected geographic features, not isolated blue cells. The primary river has a continuous visual core and belongs to canonical river data. It must remain readable through forest, beach, and winter snow.

Mountains

Mountains are connected formations with a visible base or ridge and spaced larger landmarks. They must not look like randomly scattered trees. High-terrain membership is determined independently from seasonal snow appearance.

6. Seasons and weather

Seasons describe the calendar. Weather describes current conditions. Snow is not terrain.

Seasons: Spring, Summer, Fall, Winter

Weather: Clear, Rain, Snow, Storm, Drought

Optional later: Fog

Every retained weather state must have a clear role, visible feedback, and a consequence unless it is explicitly marked cosmetic.

Weather	Consequence	Visual signal	Player feedback
Clear	Normal production and movement	Normal seasonal treatment	Baseline state
Rain	Wet-season production or outdoor modifier	Rain and wet ground	Weather change message
Snow	Winter cold or work/farm modifier if designed	White ground and snowfall	Snow-start message
Storm	Building damage and severe weather	Heavy rain, wind, flashes	Warning and damage feedback
Drought	Reduced farm output and ecological stress	Dry ground and vegetation	Start notification and active effect
Fog	Reduced visibility, or cosmetic only	Layered haze	Explain consequence if gameplay-affecting

Rain is more likely in spring and fall. Summer favors clear weather, with drought becoming more likely after a hot and dry period. Winter favors snow and clear frost; winter rain can represent a

thaw. Storms are occasional and dangerous. Fog is postponed until it can be made useful or convincing.

7. Snow as a weather layer

Snow is derived from winter and weather state. It does not change logical terrain, buildability, collision, movement, or production categories.

```
Grassland + winter → snowy grass  
Forest + winter → snowy ground and tree caps  
Beach + winter → light frost or thin snow  
River + winter → blue river with frosted banks  
Mountain + winter → snow on peaks and upper ledges
```

Snow has two visual positions:

1. Ground snow below props, buildings, and actors.
2. Accumulation caps above trees, roofs, rocks, and mountain peaks.

All snow uses masks matching the visible sprite footprint. Transparent corners of an isometric PNG must remain transparent. A full white rectangle is not acceptable.

The first version may use render-only coverage derived from season and active weather. Gradual accumulation and melting can be added later without changing the logical terrain model.

8. Rendering architecture

The prototype should use fixed canonical sprite orientation and one consistent camera transform. The target layer order is:

```
map background
→ base visual terrain
→ shoreline transitions
→ river formations
→ mountain formations
→ ground snow
→ props and trees
→ snow caps
→ buildings and construction previews
→ people and animals
→ rain, snow, storm, and seasonal particles
→ drought grade, fog, and atmospheric effects
→ UI
```

Static terrain and formation layers are cached separately from dynamic weather. Weather particles update every frame, but weather changes must not rebuild all static terrain. Seasonal grading is a lightweight layer, not a reason to dispose every terrain chunk.

9. Prototype implementation stages

Stage 1: Blank-grid foundation

Create a blank logical grid and a simple camera. Verify that world coordinates, screen coordinates, zoom, and formation footprints align correctly.

Stage 2: Geographic terrain

Render grass, water, beach, one coherent shoreline, one connected river, and one clustered mountain range. Do not add every asset yet.

Stage 3: Props and buildings

Add trees, rocks, one building, construction preview, and actors. Confirm that large visuals remain aligned with logical cells and gameplay footprints.

Stage 4: Snow

Add masked world-space snow ground and selective snow caps. Test clear winter, active snowfall, and winter rain/thaw.

Stage 5: Weather consequences

Add seasonal weather probabilities, drought production penalties and notifications, storm damage and warnings, and rain effects. Keep fog disabled or cosmetic until its purpose is defined.

Stage 6: Performance

Measure terrain build, formation build, weather-layer update, particle update, and frame rendering separately. Optimize invalidation and sprite management before reducing visual quality.

Stage 7: Wilderfolk adapter

Only after the prototype is visually and technically convincing should an adapter be written to consume Wilderfolk `RenderSnapshot` data. The adapter translates real logical terrain and weather state into prototype visual formations. It does not copy the old renderer internals.

10. Integration contract

The future integration boundary is data-based:

Input

- seed or map identity
- map dimensions
- logical terrain cells
- river and mountain formation data
- season
- weather
- entity/building render data

Output

- visual terrain formations
- sprite placements and masks
- snow and seasonal layers
- particle and atmosphere state
- mapping from visual formations to logical cells

The prototype must never require the old renderer's private caches, terrain switches, or Canvas-specific assumptions.

11. Acceptance criteria

The prototype is successful when:

- The visible map reads as a coherent geographic landscape.
- The ten-pixel logical grid remains invisible during normal play.
- The Mountainous preset provides sufficient buildable starting space.
- Mountains appear clustered and connected rather than scattered.
- Rivers are continuous and remain readable through different terrain.
- Grass, forest, beach, shallow water, river, and deep water are distinguishable.
- Winter makes the map visibly white without hiding rivers or creating square snow halos.
- Drought reduces farm output and clearly informs the player.
- Storm consequences are visible and explained.
- Fog is either meaningful or explicitly postponed.
- Camera panning and zooming preserve alignment.
- Weather changes do not rebuild static terrain unnecessarily.
- The same visual prototype can consume deterministic map data repeatedly.
- Wilderfolk can continue running unchanged while the prototype is developed.

12. Final principle

The new system should be built from scratch in a copy because the current implementation contains too many overlapping responsibilities and unused terrain concepts. The prototype should be simpler, not larger: fewer meaningful terrain categories, larger visual formations, clear seasonal weather, and one authoritative logical grid underneath.

The target is:

```
one clean visual prototype
+ one invisible logical grid
+ one terrain-definition table
+ connected geographic formations
+ weather with understandable consequences
+ snow as seasonal presentation
+ no dependence on the current renderer
+ a deliberate adapter for future Wilderfolk integration
```

This is a substantial project, but it is justified because it creates a system that can be understood, tested, compared, and improved without continuing to patch the existing terrain pipeline.